

Introduction to DBMS

What is Data?

- ❖ Data is a collection of raw, unorganized facts and details like text, observations, figures, symbols, and descriptions of things, etc.
In other words, **data does not carry any specific purpose and has no significance by itself.**
Moreover, data is measured in terms of bits and bytes – which are basic units of information in the context of computer storage and processing.
- ❖ Data can be recorded and doesn't have any meaning unless processed.

Types of Data:

- 1) **Quantitative**
 - a) Numerical form
 - b) Examples: Weight, volume, cost of an item
- 2) **Qualitative**
 - a) Descriptive, but not numerical
 - b) Examples: Name, gender, hair color of a person

What is Information?

- Information is **processed, organized, and structured data.**
- It provides **context to the data and enables decision-making.**
- It is processed data that makes sense to us.
- Information is extracted from the data by **analyzing and interpreting** pieces of data.

Example: You have data of all the people living in your locality — that's just raw data.

When you analyze and interpret it, you may conclude:

- There are 100 senior citizens.
- The sex ratio is 1.1.
- Newborn babies are 100.

These conclusions are **information.**

Data vs Information

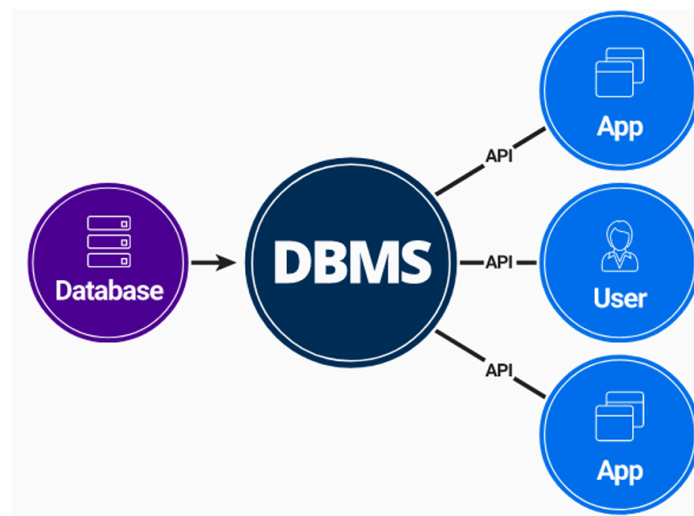
- Data is a collection of facts, while information puts those facts into context.
- While data is raw and unorganized, information is organized.
- Data points are individual and sometimes unrelated. Information maps out that data to provide a big-picture view of how it all fits together.
- Data, on its own, is meaningless. When it's analyzed and interpreted, it becomes meaningful information.
- Data does not depend on information; however, information depends on data.
- Data typically comes in the form of graphs, numbers, figures, or statistics. Information is typically presented through words, language, thoughts, and ideas.
- Data isn't sufficient for **decision-making**, but you can make decisions based on information.

What is a Database?

- A database is an electronic system where data is stored in a way that it can be easily accessed, managed, and updated.
- To make real use of data, we need **Database Management Systems (DBMS)**.

What is DBMS?

- ❖ A **Database Management System (DBMS)** is a collection of **interrelated data** and a **set of programs to access those data**. The collection of data, usually referred to as the **database**, contains information relevant to an enterprise.
The primary goal of a DBMS is to provide a way to **store and retrieve database information** that is both convenient and efficient.
- ❖ A DBMS includes the database itself along with all the software and functionality. It is used to perform different operations, such as **addition, access, updating, and deletion** of the data.



DBMS vs File Systems

File-processing systems have major disadvantages:

1. Data redundancy and inconsistency
2. Difficulty in accessing data
3. Data isolation
4. Integrity problems
5. Atomicity problems
6. Concurrent-access anomalies
7. Security problems

👉 These seven disadvantages are exactly the **advantages of using a DBMS**.

DBMS Architecture

1. View of Data (Three Schema Architecture)

- The major purpose of DBMS is to provide users with an **abstract view** of the data. That is, the system hides certain details of how the data is stored and maintained.
- To simplify user interaction with the system, abstraction is applied through several levels of abstraction.
- The main objective of three-level architecture is to enable multiple users to access the same data with a personalized view while storing the underlying data only once.

a. Physical level / Internal level:

- The lowest level of abstraction describes how the data are stored.
- Low-level data structures used.
- It has a **physical schema** which describes the physical storage structure of the DB.
- Talks about: Storage allocation (e.g., N-ary tree), Data compression & encryption, etc.

Goal: Define algorithms that allow efficient access to data.

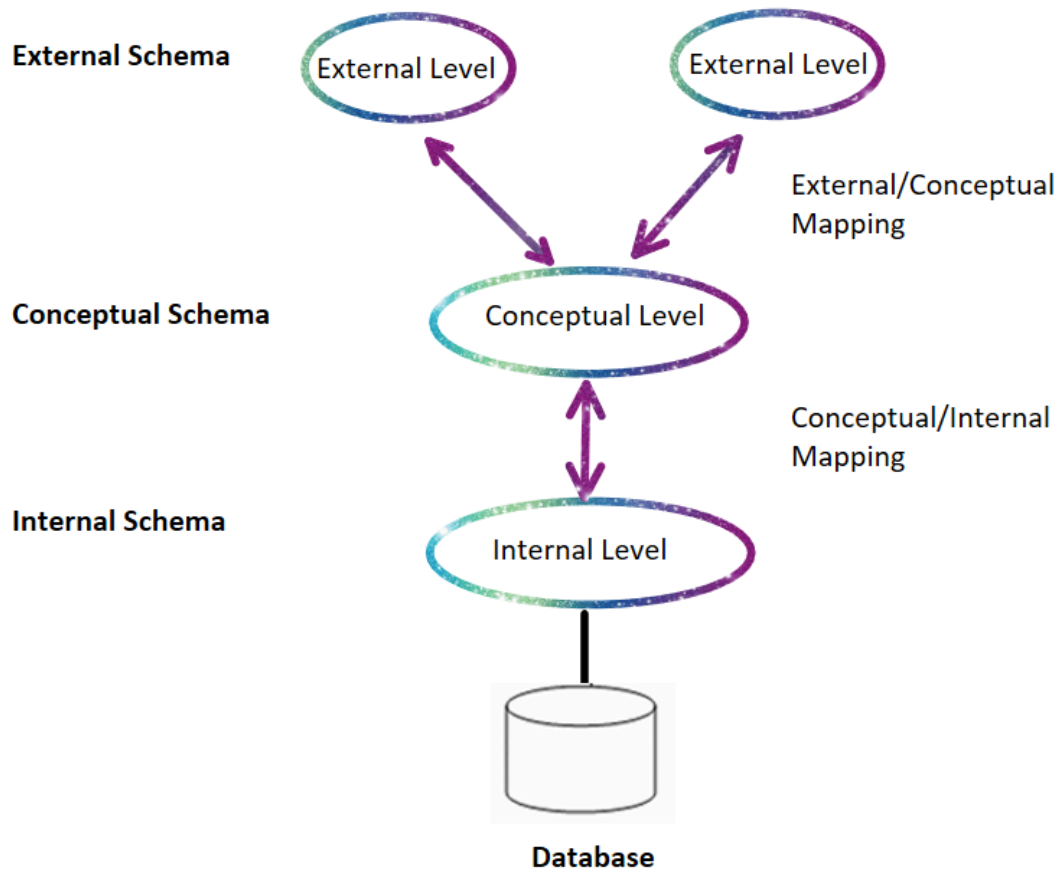
b. Logical level / Conceptual level:

- The **conceptual schema** describes the design of a database at the conceptual level. Describes **what** data are stored in DB, and what **relationships** exist among those data.
- A user at the logical level does not need to be aware of physical-level structures.
- The **Database Administrator (DBA)** is the one who must decide what information to keep in the DB and uses the logical level of abstraction.

Goal: Ease of use.

c. View level / External level:

- Highest level of abstraction; aims to simplify users' interaction with the system by providing different views to different end-users.
- Each **view schema** describes the database part that a particular user group is interested in and hides the remaining database from that user group.
- At the external level, a database contains several schemas that are sometimes called **subschemas**. Subschemas describe different views of the database.
- Views also provide a **security mechanism** to prevent users from accessing certain parts of the DB.



2. Instances and Schemas

- The collection of information stored in the DB at a particular moment is called an **instance** of the database.
- The overall design of the DB is called the **DB schema**.
- Schema is a structural description of data. Schema **doesn't change frequently**, but data may change often.
- **DB schema** corresponds to variable declarations (along with types) in a program.
- We have 3 types of schemas: **Physical**, **Logical**, and several **View schemas (subschemas)**.
- Logical schema is most **important** in terms of its effect on application programs, as programmers construct applications by using the logical schema.
- **Physical data independence:** Physical schema changes should not affect the logical schema or application programs.

3. Data Models

- Provides a way to describe the **design** of a DB at the logical level.
- Underlying the structure of the DB is the **Data Model**: a collection of conceptual tools for describing **data, data relationships, data semantics, and consistency constraints**.
- Examples: ER Model, Relational Model, Object-Oriented Model, Object-Relational Data Model, etc.

4. Database Languages

- **Data Definition Language (DDL):** Used to specify the database schema.
- **Data Manipulation Language (DML):** Used to express database queries and updates.
- Practically, both DDL and DML features are present in a single DB language, e.g., SQL.

DDL:

Used to define schemas. Specifies consistency constraints, which must be checked every time the DB is updated.

DML:

- Data manipulation involves:
 1. Retrieval of information stored in DB
 2. Insertion of new information into DB
 3. Deletion of information from the DB
 4. Updating existing information stored in DB
- **Query language**, a part of DML, is used to request retrieval of specific information.

5. How is Database Accessed from Application Programs?

- Applications (written in host languages like C/C++, Java) interact with the DB.
- Example: A banking system's module generating payrolls accesses the DB by executing DML statements from the host language.
- APIs are provided to send DML/DDL statements to the DB and retrieve results.

APIs:

- **ODBC (Open Database Connectivity)** – for languages like C
 - **JDBC (Java Database Connectivity)** – for Java
-

6. Database Administrator (DBA)

1. A person who has **central control** of both the data and the programs that access those data.
2. **Functions** of DBA.
 - I. Schema Definition
 - II. Storage structure and access methods
 - III. Schema and physical organization modifications
 - IV. Authorization control
 - V. Routine Maintenance:
 - Periodic backups
 - Security patches
 - Any upgrades

7. DBMS Application Architectures

Client machines, on which remote DB users work, and server machines on which the DB system runs.

a. T1 Architecture

- The client, server, and DB all present on the same machine.

b. T2 Architecture

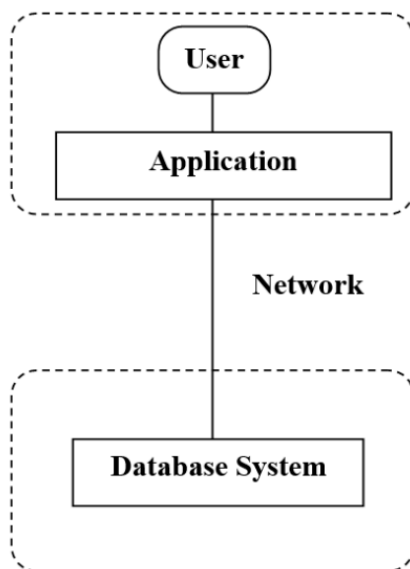
- Application is partitioned into 2 components.
- Client machine invokes DB system functionality at the server end through query language statements.
- API standards like **ODBC and JDBC** are used for client-server interaction.

c. T3 Architecture

- Application is partitioned into 3 logical components.
- Client machine is just a frontend and doesn't contain any direct DB calls.
- Client communicates with App Server → App Server communicates with DB System to access data.
- Business logic, what action to take at the condition is in the App Server itself.
- T3 Architecture is best suited for **WWW (web-based) applications**.

Advantages of T3 Architecture:

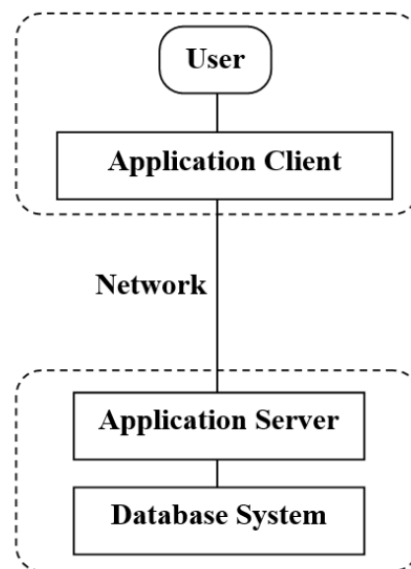
1. **Scalability** due to distributed application servers
2. **Data Integrity**: App Server acts as a middle layer between client and DB, minimizing chances of data corruption
3. **Security**: Client cannot directly access the DB, making it more secure.



(a) Two-tier Architecture

client

Server



(a) Three-tier Architecture

Entity-Relationship Model

1. Data Model

- A collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.

2. ER Model

- It is a high-level data model based on a perception of the real world that consists of a collection of basic objects, called **entities**, and of **relationships** among these objects.
- The graphical representation of the ER model is the **ER diagram**, which acts as a blueprint of the database.
- It has a physical existence.

3. Entity

- An **entity** is a “thing” or “object” in the real world that is distinguishable from all other objects.
- Example: Each student in a college is an entity.
- An entity can be uniquely identified (by a primary attribute, a.k.a. **Primary Key**).

Types of Entities:

- **Strong Entity**: Can be uniquely identified.
- **Weak Entity**: Can't be uniquely identified; depends on a strong entity.
 - Doesn't have sufficient attributes to select a uniquely identifiable attribute.
 - Example: Loan → Strong Entity, Payment → Weak Entity (as installments are just counters generated per loan).
 - A weak entity depends on a strong entity for existence.

4. Entity Set

- A set of entities of the same type that share the same properties or attributes.
- Examples:
 - Student is an entity set.
 - Customer of a bank.

5. Attributes

- An entity is represented by a set of **attributes**.
- Each entity has a value for each of its attributes.
- For each attribute, there is a **domain** (or value set).

Example (Student Entity):

- Student_ID
- Name
- Standard
- Course
- Batch
- Contact Number
- Address

Types of Attributes

a. Simple Attributes

- Cannot be divided further.
- Example: Account Number, Roll Number.

b. Composite Attributes

- Can be divided into subparts.
- Example:
 - Name → First Name, Middle Name, Last Name
 - Address → Street, City, State, PIN Code

c. Single-Valued Attributes

- Hold only one value.
- Example: Student ID, Loan Number

d. Multi-Valued Attributes

- Can have more than one value.
- Examples: Phone Numbers, Nominee Names, Dependent Names
- May have limit constraints (upper or lower limits)

e. Derived Attributes

- Value is derived from other attributes.
- Examples: Age (from DOB), Loan Age, Membership Period

f. NULL Value

- An attribute takes a **NULL** value when an entity does not have a value for it.
- It may indicate:
 - **Not applicable** (e.g., no middle name)
 - **Unknown** (e.g., salary not updated yet)

6. Relationships

- **Associations** among two or more entities.
- Examples:
 - Person has Vehicle
 - Parent has Child
 - Customer borrows Loan

Types of Relationships:

- **Strong Relationship:** Between two independent entities.
- **Weak Relationship:** Between a weak entity and its strong entity.
 - Example: Loan Payment

Degree of Relationship

- Number of entities participating in a relationship.
- **Binary relationships** are the most common.

Degree	Description	Example
Unary	One entity participates	Employee manages Employee
Binary	Two entities participate	Student takes Course
Ternary	Three entities participate	Employee works on Job at Branch

7. Relationship Constraints

a. Mapping Cardinality (Cardinality Ratio)

Defines how many entities of one set are associated with entities of another set.

1. **One-to-One (1:1)**
 - Example: Citizen ↔ Aadhar Card
2. **One-to-Many (1:N)**
 - Example: Citizen → Vehicle
3. **Many-to-One (N:1)**

- Example: Course $\leftarrow$ Professor
- 4. **Many-to-Many (M:N)**
 - Examples: Customer $\leftrightarrow$ Product, Student $\leftrightarrow$ Course

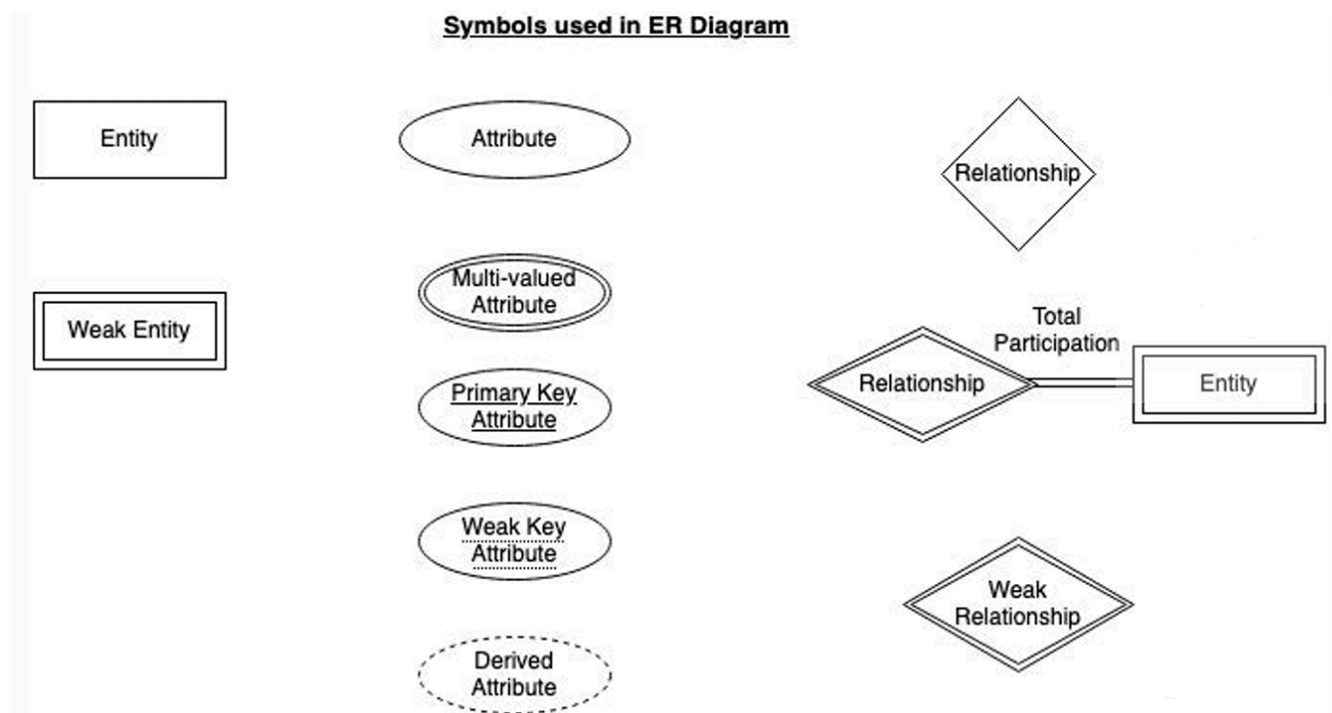
b. Participation Constraints (Minimum Cardinality)

- **Partial Participation:** Not all entities are involved in the relationship instance.
- **Total Participation:** Each entity must be involved in at least one relationship instance.

Examples:

- Customer borrows Loan:
 - **Loan** has **Total Participation** (can't exist without Customer)
 - **Customer** has **Partial Participation**
- **Weak entities always have total participation;** strong entities may or may not.

8. ER Notations



Extended ER Features

1. Why Extended ER?

- Basic ER features studied in LEC-3 can model most DB features.
- However, when **complexity increases**, it is better to use **Extended ER features** to model the database schema more effectively.

2. Specialisation

- In the ER model, we may need to **subgroup an entity set** into smaller entity sets that are distinct in some way.
- **Specialisation** is the process of **splitting** an entity set into sub-entity sets based on their **functionality, specialties, or features**.
- It follows a **Top-Down approach**.

Example:

- The **Person** entity set can be divided into **Customer**, **Student**, and **Employee**.
 - **Person** is the **superclass**, and others are **subclasses**.
 - There exists an **"is-a" relationship** between the superclass and the subclasses.
 - This relationship is depicted using a **triangle** in the ER diagram.

Why Specialisation?

1. Some attributes may be applicable only to a few entities in the parent entity set.
2. Helps designers **highlight unique characteristics** of sub-entities.
3. Improves the clarity and **refinement of the DB design**.

3. Generalisation

- **Generalisation** is the **reverse of specialisation**.
- When a DB designer notices **overlapping properties** in different entity sets, they may **combine them** into a **new generalised (superclass) entity set**.
- There is still an **"is-a" relationship**, now from **subclass to superclass**.
- Follows a **Bottom-Up approach**.

Example:

- **Car**, **Jeep**, and **Bus** have overlapping attributes.
- These can be **generalised** into a new entity set called **Vehicle**.

Why Generalisation?

1. Makes the DB design **more refined and simpler**.
2. **Eliminates duplication** of common attributes.

4. Attribute Inheritance

- Both **specialisation** and **generalisation** involve **attribute inheritance**.
- Attributes of **higher-level entity sets** are **inherited** by their **lower-level** entity sets.

Example:

- **Customer** and **Employee** inherit all attributes of the **Person** entity.

5. Participation Inheritance

- If a **parent entity** participates in a relationship, then its **child entities** also participate in that relationship.
- This maintains **consistency** across related entities.

6. Aggregation

- **How do we show relationships among relationships?**
→ The answer is **Aggregation**.
- Aggregation allows you to **treat a relationship as a higher-level (abstract) entity**.
- It helps:
 1. **Reduce redundancy**
 2. **Model more complex situations**
 3. Represent **"has-a" relationships** involving a relationship

Relational Model

1. Overview

- **Relational Model (RM)** organizes data in the form of **relations (tables)**.
- A **relational database** consists of a **collection of tables**, each with a **unique name**.
- A **row (tuple)** in a table represents a **relationship among a set of values**.
- The **table** is a collection of such relationships.

2. Basic Terminology

- **Tuple**: A single row of the table representing a single data point / unique record.
- **Columns**: Represent the **attributes** of the relation.
 - Each attribute has a set of permitted values known as the **domain**.
- **Relation Schema**: Describes the **structure** of the relation.
 - Includes the name of the relation and all the column names.

3. RDBMS Examples

- Popular RDBMS systems include: **Oracle, IBM DB2, MySQL, MS Access**

4. Table Properties

- **Degree** of a table: Number of **attributes/columns** in the relation.
- **Cardinality**: Total number of **tuples/rows** in a relation.

5. Relational Key

- A **set of attributes** that can **uniquely identify each tuple** in a relation.

6. Properties of a Table (Relational Model)

1. Name of each relation must be **distinct**.
2. Values in the table must be **atomic** (indivisible).
3. Name of each attribute must be **unique**.
4. Each **tuple must be unique**.
5. **Order** of rows and columns has **no significance**.
6. Tables must follow **integrity constraints** to ensure consistency.

7. Types of Keys in Relational Model

1. **Super Key (SK)**: Any combination of attributes that can **uniquely identify** each tuple.
2. **Candidate Key (CK)**: A **minimal** Super Key (no redundant attributes).

- Must have **non-null values**.
- 3. **Primary Key (PK):**
 - Selected from Candidate Keys.
 - Typically has the least **number of attributes**.
- 4. **Alternate Key (AK):**
 - All **Candidate Keys** except the chosen **Primary Key**.
- 5. **Foreign Key (FK):**
 - Used to create relationships between two tables.
 - A relation **r1** includes the **Primary Key** of another relation **r2**.
 - In this context:
 - **r1** → **Referencing (Child)** relation
 - **r2** → **Referenced (Parent)** relation
 - FK helps with **cross-referencing** between relations.
- 6. **Composite Key:** A Primary Key made up of **two or more attributes**.
- 7. **Compound Key:** A Primary Key **composed of Foreign Keys**.
- 8. **Surrogate Key:**
 - A **synthetic key** generated automatically by the DB.
 - Usually an **integer**.
 - Can be used as a **Primary Key**.

8. Integrity Constraints

- CRUD operations must comply with **integrity policies** to keep the DB **consistent**.
- Prevents **accidental corruption** of the database.

a. Domain Constraints

- Restrict the type and format of data stored in an attribute.
- Examples:
 - Enforcing a date range (e.g., Birth Year < 2002)
 - Restricting data type (e.g., **Age** should be an integer)

b. Entity Constraints

- Every relation should have a **Primary Key**.
- **Primary Key ≠ NULL**

c. Referential Constraints

- Specified between two relations & helps maintain consistency among tuples of two relations.
- It requires that the value appearing in specified attributes of any tuple in the referencing relation also appear in the specified attributes of at least one tuple in the referenced relation.

- If FK in the referencing table refers to PK of referenced table then every value of the FK in referencing table must be NULL or available in referenced table.
- FK must have the matching PK for its each value in the parent table or it must be NULL.

9. Key Constraints (Types)

Constraint	Description
NOT NULL	Ensures no null values; every record must have a value.
UNIQUE	Ensures all values in a column are different.
DEFAULT	Assigns a default value if none is provided during insertion.
CHECK	Verifies a condition before accepting data.
PRIMARY KEY	Uniquely identifies each record; must be NOT NULL + UNIQUE .
FOREIGN KEY	Ensures a valid link exists between two related tables.

Transform – ER Model to Relational Model

1. Introduction

- Both ER-Model and Relational Model are abstract logical representations of real-world enterprises.
- Because the two models imply similar design principles, we can convert **ER design** into **Relational design**.
- Converting a DB representation from an **ER diagram to a table format** is how we arrive at **Relational DB-design** from an ER diagram.

2. ER Diagram Notations to Relations

1. Strong Entity

1. Becomes an individual table with entity name; attributes become columns of the relation.
2. Entity's Primary Key (PK) is used as Relation's PK.
3. Foreign Keys (FK) are added to establish relationships with other relations.

2. Weak Entity

1. A table is formed with all the attributes of the entity.
2. PK of its corresponding Strong Entity will be added as FK.
3. The PK of the relation will be a **composite PK**: {FK + Partial Discriminator Key}.

3. Single-Valued Attributes

1. Represented as columns directly in the tables/relations.

4. Composite Attributes

1. Handled by creating a separate attribute itself in the original relation for each component of the composite attribute.
2. Example:
 - **Address: {street-name, house-no}** is a composite attribute in the customer relation.
 - We add **address-street-name** and **address-house-no** as new columns and ignore "address" as a standalone attribute.

5. Multivalued Attributes

1. New tables (named after the original attribute) are created for each multivalued attribute.
2. PK of the entity is used as column FK in the new table.
3. Multivalued attribute's name is added as a column to define multiple values.

4. PK of the new table: {FK + multivalued name}.
5. Example:
 - For a strong entity **Employee**, **dependent-name** is a multivalued attribute.
 - New table: **dependent-name** with columns **emp-id**, and **dname**.
 - PK: {emp-id, name}
 - FK: {emp-id}

6. Derived Attributes

- **Not considered** in the tables.

7. Generalisation

Method 1:

1. Create a table for the higher-level entity set.
2. For each lower-level entity set, create a table that includes:
 - A column for each of its attributes
 - Plus a column for each attribute of the **Primary Key** of the higher-level entity set

Example:

- Banking System: Generalisation of Account → Savings & Current
- Table 1: **account** (**account-number**, **balance**)
- Table 2: **savings-account** (**account-number**, **interest-rate**, **daily-withdrawal-limit**)
- Table 3: **current-account** (**account-number**, **overdraft-amount**, **per-transaction-charges**)

Method 2:

- Used when generalisation is **disjoint and complete**, i.e.:
 - No entity is a member of two lower-level entity sets
 - Every entity in the higher-level entity set is also a member of one of the lower-level sets
- In this case:
 - Do **not create a table** for the higher-level entity set
 - For each lower-level entity set, create a table that includes:
 - All its own attributes
 - All attributes of the higher-level entity set

Tables would be:

- Table 1: **savings-account** (**account-number**, **balance**, **interest-rate**, **daily-withdrawal-limit**)

- Table 2: **current-account** (**account-number**, **balance**, **overdraft-amount**, **per-transaction-charges**)

Drawbacks of Method 2:

- In overlapping generalisations, attributes like **balance** are stored **twice**, leading to redundancy
- If generalisation is **not complete**, entities that don't belong to any lower-level set can't be represented

8. Aggregation

1. A table of the **relationship set** is made
2. Attributes include:
 - Primary Keys of the **entity sets**
 - Attributes of the **aggregation set** (if any)
3. Also add any **descriptive attribute** on the relationship

Normalisation

1. Introduction

- Normalisation is a step towards **DB optimisation**.

2. Functional Dependency (FD)

1. It's a relationship between the **primary key attribute (usually)** of the relation to that of the other attribute of the relation.
2. $X \rightarrow Y$:
 - The **left side** of FD is known as a **Determinant**,
 - The **right side** is known as a **Dependent**.

3. Types of FD

1. Trivial FD

- $A \rightarrow B$ has **trivial functional dependency** if **B is a subset of A**.
- $A \rightarrow A$, $B \rightarrow B$ are also Trivial FDs.

2. Non-trivial FD

- $A \rightarrow B$ has a **non-trivial functional dependency** if **B is not a subset of A**.
- That is, $A \cap B = \emptyset$ (empty set)

4. Rules of FD (Armstrong's Axioms)

1. Reflexive

- If 'A' is a set of attributes and 'B' is a subset of 'A', then $A \rightarrow B$ holds.
- If $A \supseteq B$ then $A \rightarrow B$.

2. Augmentation

- If B can be determined from A, then adding an attribute to this functional dependency won't change anything.
- If $A \rightarrow B$ holds, then $AX \rightarrow BX$ holds too. 'X' being a set of attributes.

3. Transitivity

- If A determines B and B determines C, we can say that A determines C.
- If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$.

5. Why Normalisation?

- To **avoid redundancy** in the DB and **not store duplicate data**.

6. What happens if we have redundant data?

- **Insertion, Deletion, and Updation anomalies** arise.

7. Anomalies

Anomalies mean **abnormalities**. There are **three types** introduced by data redundancy:

1. Insertion Anomaly

- When certain data (attribute) cannot be inserted into the DB **without the presence of other data**.

2. Deletion Anomaly

- Occurs when the **deletion of data results in the unintended loss** of some other important data.

3. Updation Anomaly (Modification Anomaly)

- When an update of a single data value **requires multiple rows** of data to be updated.
- Due to updating many places, **data inconsistency** may arise if one forgets to update it everywhere.
- Due to these anomalies:
 - DB size increases
 - DB performance becomes very slow
- To **rectify** these anomalies and their **effect**, we use a **database optimisation technique** called **NORMALISATION**.

8. What is Normalisation?

1. Normalisation is used to **minimise redundancy** from relations.
2. It is also used to eliminate **undesirable characteristics** like **Insertion, Update, and Deletion Anomalies**.
3. Normalisation divides:
 - Composite attributes → **individual attributes**
 - Larger table → **smaller linked tables**
4. The **normal form** is used to **reduce redundancy** from the database table.

9. Types of Normal Forms

1. First Normal Form (1NF)

- Every relation cell must have an **atomic value**.
- Relation must not have **multi-valued attributes**.
- Relation must be in **1NF**.

2. Second Normal Form (2NF)

- There should not be any **partial dependency**.
- All **non-prime attributes** must be **fully dependent** on the **Primary Key**.
- A non-prime attribute cannot depend on **part of the Primary Key**.

3. Third Normal Form (3NF)

- Relation must be in **2NF**.
- No **transitive dependency** should exist.
- A **non-prime attribute** should not determine another **non-prime attribute**.

4. Boyce-Codd Normal Form (BCNF)

- Relation must be in **3NF**.
- For any FD: $A \rightarrow B$, **A must be a super key**.
- We must not derive a **prime attribute** from any **prime or non-prime attribute**.

10. Advantages of Normalisation

1. Helps to **minimise data redundancy**.
2. Provides **greater overall database organisation**.
3. Maintains **data consistency** in the database.

Transaction

1. Transaction

1. A **transaction** is a unit of work done against the DB in a **logical sequence**.
2. Sequence is **very important** in a transaction.
3. It is a logical unit of work that contains **one or more SQL statements**. The result of all these statements in a transaction either:
 - **Completes successfully** (all changes to the database become **permanent**), or
 - **Fails and rolls back** (all the changes are **undone**)

2. ACID Properties

To ensure the **integrity of data**, the DBMS must maintain the following **ACID properties** of a transaction:

1. Atomicity

- Either **all operations** of the transaction are reflected properly in the DB, or **none are**.

2. Consistency

- Integrity constraints must be **maintained before and after** the transaction.
- The **DB must remain consistent** after the transaction.

3. Isolation

- Even though multiple transactions may **execute concurrently**, the system guarantees that for every pair of transactions **T_i** and **T_j**, it appears to **T_i** that:
 - Either **T_j finished** before **T_i** started, or
 - **T_j started** after **T_i** finished.
- Thus, **each transaction is unaware** of other concurrent transactions.
- Multiple transactions can happen in **isolation**, without interfering with one another.

4. Durability

- Once a transaction completes **successfully**, the **changes it has made persist**, even if there is a system failure.

3. Transaction States

1. Active State

- The **first state** in the lifecycle of a transaction.
- All **read and write operations** are being performed.

- If operations execute **without error**, it moves to **Partially Committed**.
- If any error occurs, it moves to **Failed** state.

2. Partially Committed State

- After the transaction executes, changes are saved in the **buffer (main memory)**.
- If the changes are made **permanent** on the DB, the state changes to **Committed**.
- If a failure occurs, it moves to **Failed** state.

3. Committed State

- Updates are made **permanent** on the database.
- The transaction is said to be **committed**.
- **Rollback cannot** be done from this state.
- The DB achieves a **new consistent state**.

4. Failed State

- When the transaction is executing and a **failure occurs**, making it **impossible to continue** execution.

5. Aborted State

- After reaching the **Failed** state, all changes in the buffer are **reversed**.
- The transaction is **rolled back completely**.
- The DB returns to the state **prior to the transaction**.

6. Terminated State

- A transaction is said to be **terminated** if it has either **committed** or **aborted**.

How to Implement Atomicity and Durability in Transactions

1. Recovery Mechanism Component of DBMS

- Supports **atomicity** and **durability**.

2. Shadow-Copy Scheme

1. Based on making copies of the DB (also called shadow copies).
2. Assumes only one transaction (T) is active at a time.
3. A pointer called **db-pointer** is maintained on the disk; it always points to the **current copy of the DB**.
4. When a transaction wants to update the DB, it first **creates a complete copy** of the DB.
5. All updates are made on the **new copy**, keeping the **original (shadow copy)** untouched.
6. If the transaction is **aborted**, the system **deletes the new copy**, and the old copy remains unaffected.
7. If the transaction **succeeds**, the system commits it by:
 - Ensuring all pages of the new DB copy are written to disk.
 - Updating the **db-pointer** to point to the new DB copy.
 - Deleting the old copy.
 - The transaction is **committed** at the point where **db-pointer** is written to disk.

8. Atomicity

1. If a transaction fails **before** the **db-pointer** is updated:
 - The original DB remains unchanged.
 - The transaction can be **aborted** by simply deleting the new copy.
2. Therefore, **either all updates are reflected, or none**.

9. Durability

1. If the system fails **before** the updated **db-pointer** is written:
 - On restart, the system reads the **original** DB (no transaction effect).
2. If the system fails **after** the **db-pointer** is updated:
 - The **new DB copy** is read, with all changes preserved.
3. Transaction is assumed **successful** only when **db-pointer** is updated.

10. Implementation Detail

- Relies on **atomic write** to the **db-pointer**.
- Most disk systems provide atomic writes for entire blocks or sectors.
- Ensure **db-pointer** lies **entirely in one sector**, preferably **at the beginning of a block**.

11. Disadvantage of Shadow-Copy

- **Inefficient**: Entire DB is copied for **every transaction**.

12. Log-Based Recovery Methods

1. A **log** is a sequence of records maintained in **stable storage**.
2. Any DB operation is recorded in the log **before** execution.
3. The log ensures recovery after system failure.
4. Stable storage guarantees **atomicity** for write operations and survives some failures.

13. Deferred DB Modifications

1. Ensures atomicity by **recording all DB modifications in the log** but **deferring execution** of write operations until the transaction's final action.
2. Steps:
 - All changes are logged but **not applied immediately**.
 - If the transaction **fails or aborts**, the log is ignored.
 - If the transaction **completes**, log records are used to **apply the changes**.
 - If failure occurs during this application, we perform a **redo**.

14. Immediate DB Modifications

1. DB modifications are written to the DB **while the transaction is still active**.
2. These are **uncommitted modifications**.
3. To maintain atomicity:
 - Use the **old value** from the log to **undo** if failure occurs.
 - Log records must be written to **stable storage** before the actual update.
4. Handling Failures:
 - If the system fails **before the transaction** completes or is **aborted** → use **undo** (old values).
 - If the system fails **after the transaction** completes → use **redo** (new values).

Indexing in DBMS

1. What is Indexing?

- Indexing is used to **optimize performance** by minimizing the number of disk accesses required during query processing.
- It is a **data structure** used to quickly **locate and access data** in a database table.
- Primarily helps with **read operations** like **SELECT** queries, **WHERE** clauses, etc.
- **Search Key**: A copy of the **primary key**, **candidate key**, or any other attribute used for searching.
- **Data Reference**: A pointer holding the **disk block address** where the value of the corresponding key is stored.
- Indexing is **optional**, acts as a **secondary access method**, and increases access speed.
- The **index file is always sorted**.

2. Indexing Methods

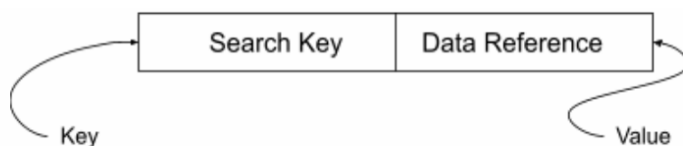
2.1. Primary Index (Clustering Index)

- A file may have several indices, on different search keys.
- If the data file is **sequentially ordered**, a **primary index** is one where the **search key also defines the sequential order** of the file.
- ⚠ Note: The term "primary index" is sometimes used to refer to an index on a **primary key**, but that is non-standard usage.

3. Dense and Sparse Indices

3.1. Dense Index

- Contains an **index record for every search-key value** in the data file.
- Each index record holds:
 - The **search key**.
 - A **pointer to the first data record** with that value.
- Remaining records with the same search key are stored **sequentially**.
- Requires **more storage space**.



3.2. Sparse Index

- Contains **index records only for some search-key values**.
- A **range of index values** maps to the **same data block**.
- Saves space compared to dense indexing.
- When accessed, **the block address is fetched**, and then **linear search** is used within the block.

4. Primary Indexing Based on Attribute Type

4.1. Based on Key Attribute

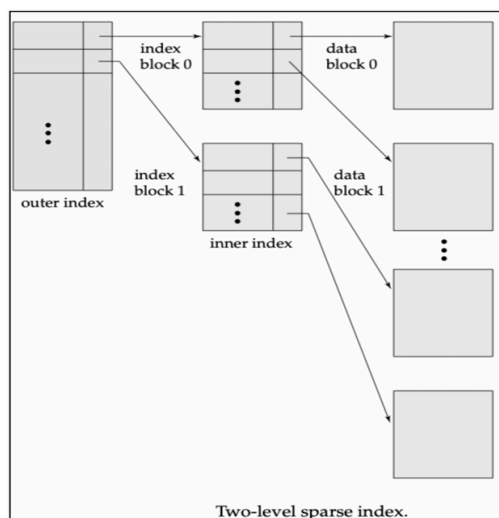
- Data file is sorted **w.r.t. primary key**.
- The **primary key** becomes the **search key**.
- A **sparse index** is created:
 - **#Entries in index file = #Blocks in data file**

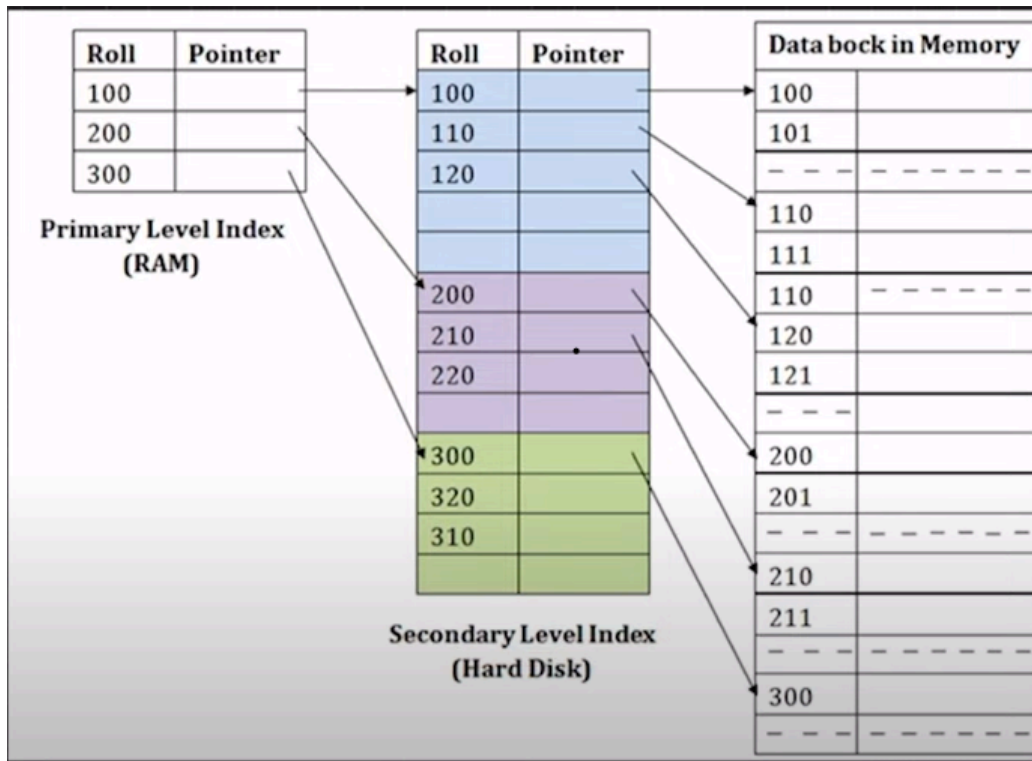
4.2. Based on Non-Key Attribute

- Data file is sorted **w.r.t. a non-key attribute**.
- Creates a **dense index**:
 - **#Entries in index file = Unique values of the non-key attribute**
- Example:
 - A company recruits employees in various departments.
 - **Clustering indexing** is used for employees **grouped by department**.

5. Multi-Level Index

- Used when a **single-level index becomes too large** for efficient binary search.
- Break down the index into **multiple levels**:
 - Top level points to lower-level indices.
 - Lower levels point to actual data blocks.





6. Secondary Index (Non-Clustering Index)

- Applicable when the **data file is unsorted** (i.e., primary indexing is not possible).
- Can be applied to both **key and non-key attributes**.
- Called **secondary** because a primary index usually already exists.
- **#Entries in index file = #Records in data file**
- This is an example of a **dense index**.

7. Advantages of Indexing

1. **Faster access and retrieval** of data.
2. Reduces **I/O operations**.

8. Limitations of Indexing

1. Requires **additional storage** for the index table.
2. **Decreases performance** for **INSERT**, **DELETE**, and **UPDATE** queries due to index updates.

NoSQL

1. What is NoSQL?

- NoSQL databases (aka "**Not Only SQL**") are **non-tabular** and store data differently than relational tables.
- Main types: **Document**, **Key-Value**, **Wide-Column**, and **Graph**.
- **Schema-free**, flexible, and scalable with large data volumes and high user loads.
- Characteristics:
 1. Schema-free.
 2. Flexible, non-tabular structures.
 3. Handle massive (big) data volumes.
 4. Mostly open-source with **horizontal scaling** support.
 5. Stores data in **non-relational formats**.

2. History Behind NoSQL

- Emerged in the **late 2000s**, driven by:
 1. Cheaper storage → less need for tight normalisation.
 2. Rise of **unstructured data** → static schemas were costly.
 3. Developers wanted **flexibility and speed** in iteration.
 4. **Cloud computing** and distributed systems became common.
 5. Tools like **MongoDB** supported sharding, replication, and resilience.

3. NoSQL Database Advantages

A. Flexible Schema

- No need for rigid schema definitions like in RDBMS.
- Schema changes are easier.

B. Horizontal Scaling

- Scale-out using **sharding** or **replica sets**.
- Easier distribution of data across multiple servers.

C. High Availability

- Supports **auto-replication**.
- Data is stored on **multiple servers** → access continues during failures.

D. Fast Insert and Read Operations

- No joins required (in most NoSQL types).

- Data accessed together is stored together → **faster queries**.
- ⚠ Delete and update operations may be **costly**.

E. Caching Mechanism

- Built-in or easy-to-integrate caching using key-value principles.

F. Optimized for Cloud Applications

- Ideal for **agile**, cloud-native, and microservice-based systems.

4. When to Use NoSQL?

- Fast-paced agile development.
- Need to store **structured + semi-structured** data.
- High volume data ingestion.
- Requirement for **scale-out architecture**.
- Real-time applications, streaming, microservices.

5. NoSQL Misconceptions

A. No Support for Relationships

- ❌ False. Relationships can be modeled using **embedded structures** (e.g., documents within documents).

B. No ACID Transactions

- ❌ False. Some NoSQL DBs like **MongoDB** support ACID.

6. Types of NoSQL Data Models

1. Key-Value Stores

- Stores key-value pairs like a **dictionary**.
- Use cases: shopping carts, session data, caching.
- Examples: **Redis, Amazon DynamoDB, Oracle NoSQL**.
- Efficient for **constant time** lookups.
- Ideal for:
 - Real-time random data access (e.g., gaming, finance).
 - Key-based caching.
 - Simple queries.

2. Column-Oriented (Columnar / C-Store / Wide-Column)

- Stores data by **columns**, not rows.

- Great for **analytics**, where only some columns are queried.
- Compresses efficiently.
- Use cases: time series data, OLAP systems.
- Examples: **Cassandra**, **Redshift**, **Snowflake**.

3. Document-Based Stores

- JSON-like format.
- Fields can include arrays, nested documents, etc.
- ACID-compliant in some databases.
- Use cases: e-commerce, mobile apps.
- Examples: **MongoDB**, **CouchDB**.

4. Graph-Based Stores

- Nodes (data) and edges (relationships).
- Efficient for traversing deeply connected data.
- Use cases: fraud detection, social networks, knowledge graphs.
- Examples: **Neo4j**, **Amazon Neptune**.

7. NoSQL Disadvantages

1. **Data Redundancy**
 - Optimized for reads, not for eliminating duplicates.
2. **Update/Delete are Costly**
 - Data duplication can make updates complex.
3. **No One-Size-Fits-All**
 - Different NoSQL types solve **different** problems.
4. **Weak ACID Support**
 - Not all NoSQL DBs support full transactional integrity.
5. **Limited Data Constraints**
 - Schema flexibility means weaker consistency checks.

8. SQL vs NoSQL Comparison

Feature	SQL	NoSQL
Storage Model	Tables with fixed rows/columns	Document (JSON), Key-Value, Wide Column, Graph

Development History	1970s, reducing duplication	2000s, built for scalability and fast iteration
Examples	Oracle, MySQL, SQL Server, PostgreSQL	MongoDB, CouchDB, Redis, DynamoDB, Cassandra, Neo4j
Use Cases	General-purpose	Document: general, KV: lookup, Column: analytics, Graph: relationships
Schema	Fixed	Flexible
Scaling	Vertical (scale-up)	Horizontal (scale-out)
ACID Support	Full	Partial (MongoDB supports full ACID)
JOINS	Typically required	Typically not required
ORM Mapping	Required	Often unnecessary (natural mapping in code)

Types of Databases

1. Relational Databases

- Based on the Relational **Model**.
- Popular since the 1970s and widely used today.
- Known as **Relational Database Management Systems (RDBMS)**.
- Use **Structured Query Language (SQL)** for operations like CRUD (Create, Read, Update, Delete).
- Data is stored in **discrete tables**.
- Tables can be **JOINED** using foreign keys.

Example:

- A **User** table linked to a **Purchases** table by a **user_id** key.

Examples of RDBMS:

- **MySQL, Microsoft SQL Server, Oracle**

Features:

1. Ubiquitous with a steady user base.
2. Highly optimized for **structured data**.
3. Support **normalization** for reducing redundancy.
4. Use of standard **SQL**.
5. Limited **horizontal scalability**.
6. As data grows, systems become **complex to manage**.

2. Object-Oriented Databases

- Based on the **Object-Oriented Programming (OOP)** paradigm.
- Support:
 - **Inheritance**
 - **Object-identity**
 - **Encapsulation**
 - Rich type system (structured + collection types)
- Differs from the E-R model through encapsulation and identity.

Key Points:

- Data is stored and retrieved as **objects**.
- All relevant data is accessed as one object package, not split across tables.

Advantages:

1. Fast storage and retrieval.
2. Handles **complex data types** and relationships.
3. Friendly for **real-world modelling**.
4. Compatible with **OOP languages**.

Disadvantages:

1. High complexity can slow down CRUD operations.
2. Limited community support.
3. Does not support **views** like RDBMS.

Examples:

- **ObjectDB, GemStone**

3. NoSQL Databases

- Non-tabular databases that store data differently than relational databases.
- Include various models:
 - **Document**
 - **Key-Value**
 - **Wide-Column**
 - **Graph**

Features:

1. **Schema-free**
2. Flexible, non-tabular structures
3. Scales horizontally
4. Great for **Big Data**
5. Mostly open source
6. Stores data in formats **other than relational**

4. Hierarchical Databases

- Best suited for **hierarchical data models**.

Structure:

- Tree-like organization with:
 - One **root (parent)** node
 - Multiple **child** records
- Each child can only have **one parent**.

Characteristics:

- Data is stored in fields; each field has a **single value**.
- Retrieval requires traversing from the root node.

Advantages:

1. Simple and fast for **one-to-many** relationships.
2. Works well with **drop-down menus**, **folder structures**, etc.
3. Easy addition/deletion of information.
4. Supported by most programming languages.

Disadvantages:

1. **Inflexible** structure.
2. Cannot model **many-to-many** relationships.
3. Requires **sequential search** (top-to-bottom).
4. Data redundancy due to repetitive storage.

Example:

- **IBM IMS**

5. Network Databases

- Extension of **Hierarchical databases**.
- Allows a **child record** to associate with **multiple parent records**.

Structure:

- Organized as a **graph**, not a tree.

Features:

1. Capable of modeling **complex relationships**.
2. Flexible **M:N** links.
3. Difficult to maintain.
4. Retrieval may be **slower** due to complex links.
5. Limited web community support.

Examples:

- **Integrated Data Store (IDS)**
- **IDMS (Integrated Database Management System)**
- **Raima Database Manager**
- **TurboIMAGE**

Clustering in DBMS

1. What is Database Clustering?

Database clustering, also referred to as *Replica Sets*, involves connecting multiple servers or instances to a single database system. This approach becomes essential when a single server is no longer sufficient to handle the volume of data or number of incoming requests.

2. Purpose and Use

In clustering, the same dataset is replicated across different servers. SQL is commonly used to manage this data across the cluster. This setup ensures better data availability, reliability, and performance.

3. Key Advantages of Clustering

- **Data Redundancy:**

Clustering ensures necessary data redundancy by replicating data across multiple servers. This isn't the kind of redundancy that causes anomalies—it's synchronized redundancy that ensures availability. If one server fails, the data remains accessible through other servers.

- **Load Balancing:**

Databases don't scale automatically. Clustering distributes workload among multiple servers, ensuring no single machine is overwhelmed. This improves performance and allows the system to support spikes in traffic smoothly. Load balancing is a direct contributor to high availability.

- **High Availability:**

High availability refers to the system's ability to remain accessible and operational. With clustering, even if one server goes down, others continue to serve the data. This is particularly important for systems that handle frequent transactions or analytics.

4. How Does Clustering Work?

In a clustered architecture, incoming requests are distributed among multiple nodes (servers). Each request may be executed by a different system, and if one node fails, others take over. This fault-tolerant design ensures continued service with minimal or no disruption. Features like load balancing and failover mechanisms make clustering robust and reliable.

Partitioning & Sharding in DBMS (Database Optimisation)

1. Introduction to Partitioning

Large databases can become difficult to manage and slow to process. Partitioning helps by dividing a massive database (including its data, metrics, and indexes) into smaller, more manageable pieces called **partitions**.

- These partitions can be directly accessed by SQL queries without requiring query changes.
- Data Definition Language (DDL) operations also become more efficient on these smaller parts instead of acting on the entire database at once.

2. Why Partitioning?

Partitioning distributes database objects across different servers, which enhances:

- Performance
- Scalability
- Manageability

3. When horizontal scaling is applied (adding multiple machines), it becomes difficult to maintain relationships in a traditional relational database. Partitioning makes it easier to manage large amounts of data across these servers.

Types of Partitioning

3. Vertical Partitioning

- Involves slicing the table **column-wise**.
- Different attributes (columns) are stored on different servers.
- To access a full record, multiple servers may need to be queried.

4. Horizontal Partitioning

- Involves slicing the table **row-wise**.
- Tuples (records) are divided and stored across different servers.
- Each partition holds a subset of rows independently.

When to Apply Partitioning?

Partitioning is most useful when:

- The dataset is **too large** to manage efficiently as a whole.
- The number of read/write **requests is too high**, resulting in poor response times from a single database server.

Advantages of Partitioning

1. Improved **parallelism** — multiple partitions can be queried or processed simultaneously.
2. Better **availability** — failure in one partition/server doesn't bring the whole system down.
3. Enhanced **performance** — smaller tables mean faster operations.
4. Easier **manageability** — updates, backups, and maintenance become more modular.

5. Reduced **cost** — scaling out (horizontal) is generally more cost-effective than scaling up (adding hardware).

Distributed Database

- A **distributed database** appears as a single logical system but is spread across multiple locations or servers.
- It is made possible using optimisation techniques like **Clustering**, **Partitioning**, and **Sharding**.
- The need for such systems arises from the challenges listed in the partitioning section above.

Sharding

- **Sharding** is a specific implementation of **Horizontal Partitioning**.
- Instead of storing all data on one database instance, data is split across multiple shards (i.e., separate database instances).
- A **Routing Layer** is used to direct queries to the appropriate shard that holds the required data.

Pros of Sharding

1. **Scalability** — Easily handle growing data and traffic.
2. **Availability** — Outage in one shard doesn't affect others.

Cons of Sharding

1. **Increased Complexity** — Requires a routing layer and partition mapping logic.
2. **Non-Uniformity** — May require **Re-Sharding** when one shard becomes larger than others.
3. **Poor Fit for Analytics** — Distributed data makes analytical queries slower (known as the **Scatter-Gather Problem**).

CAP Theorem

1. Overview

The CAP Theorem is a foundational concept in **Distributed Databases**, essential for designing systems that balance trade-offs between reliability, responsiveness, and data accuracy.

2. What is CAP?

CAP stands for:

- **Consistency (C):**
Every read reflects the most recent write. All nodes show the same data at any given time.
- **Availability (A):**
Every request gets a response (success or failure), even if some nodes are down.
- **Partition Tolerance (P):**
The system continues operating even if communication between some nodes is lost.

3. CAP Theorem Statement

- In the presence of a **network partition**, a distributed system can **guarantee only two out of three: Consistency, Availability, and Partition Tolerance**.
- Partition tolerance is generally non-negotiable in real distributed systems, so the trade-off typically lies between **Consistency** and **Availability**.

CAP Theorem in NoSQL & Distributed Systems

1. CA Databases (Consistency + Availability)

- **Tradeoff:** No Partition Tolerance
- CA databases deliver consistent and available responses — but only in the absence of network partitions.
- Common in **single-node** or tightly-coupled systems.

Example:

- Traditional **RDBMS** like **MySQL**, **PostgreSQL** in single-node setups or with simple replication.

✅ Note:

RDBMS systems are often considered **CA** only in **single-node setups**.

In distributed or replicated setups (e.g., master-slave), **true CA is not guaranteed**.

During a partition, multiple nodes may **incorrectly elect a new master**, resulting in a **split-brain scenario**.

This can **break consistency**, especially if two partitions accept writes independently.

Therefore, in real-world distributed setups, even traditional RDBMSs do **not fulfill CA** fully under partition.

2. CP Databases (Consistency + Partition Tolerance)

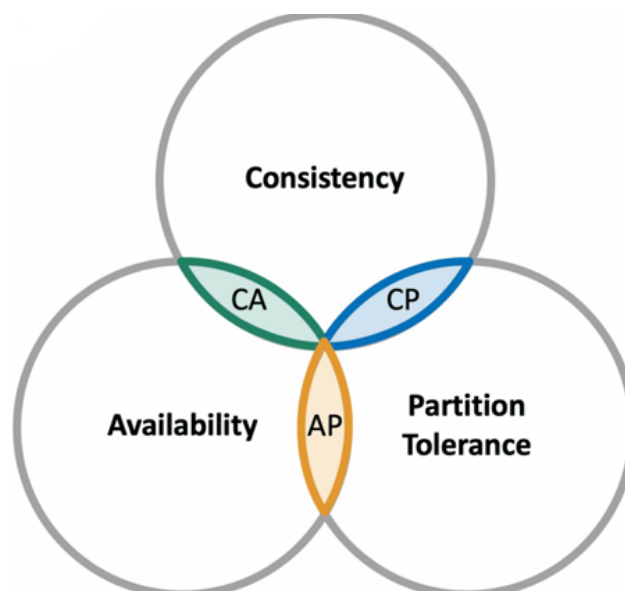
- **Tradeoff:** Reduced Availability during partition
- Prioritizes data correctness; may sacrifice availability to avoid stale or inconsistent data.
- Example: **MongoDB**
 - Writes go to a single primary.
 - Secondaries replicate the primary.
 - Failover to a secondary ensures partition tolerance with consistency.
 - Ideal for systems where **data integrity > availability** (e.g., **banking**).

3. AP Databases (Availability + Partition Tolerance)

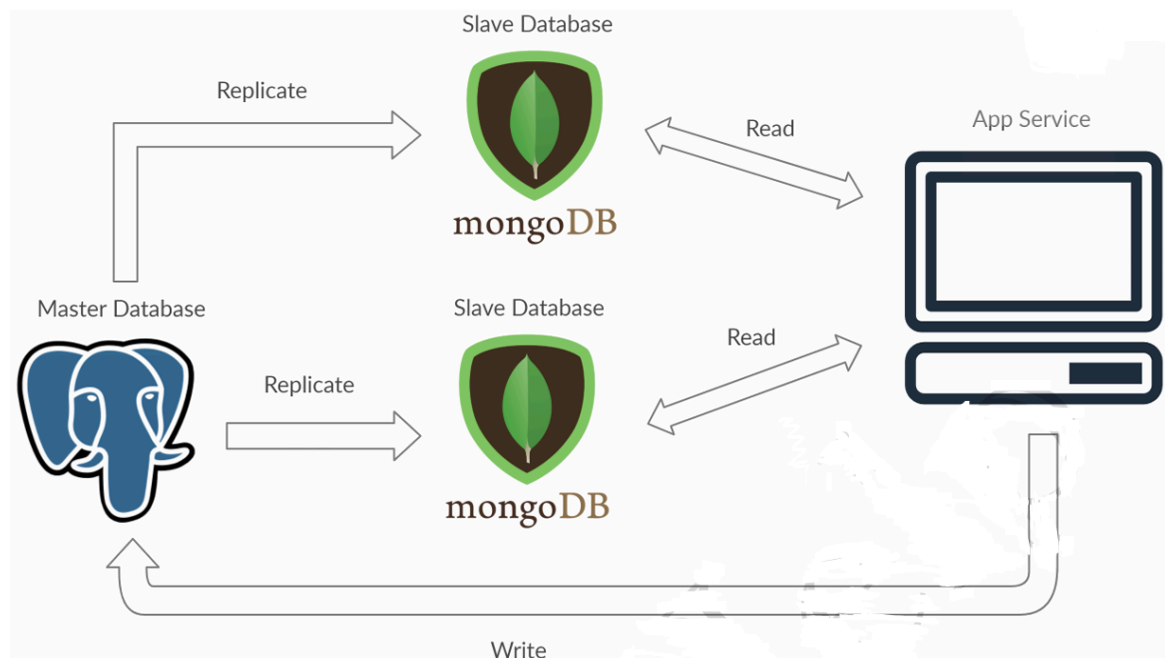
- **Tradeoff:** No immediate Consistency
- All nodes respond, even during a partition, but may serve **stale data**.
- Eventually achieve consistency via sync.
- Example: **Apache Cassandra, Amazon DynamoDB**

Conclusion

- CAP helps you understand the **fundamental trade-offs** in building distributed databases.
- Modern systems usually **require partition tolerance**, so your real design decision is between **Consistency** and **Availability**.



The Master-Slave Database Concept



1. Purpose

The Master-Slave architecture is a common database **scaling technique** used to optimize I/O performance in high-traffic systems. When a single database server becomes a bottleneck, this setup helps distribute the load efficiently.

2. Relation to Scaling Pattern

This is **Pattern 3: Command Query Responsibility Segregation (CQRS)**

- **Writes (commands)** go to the **Master**.
- **Reads (queries)** are handled by **Slave** replicas

3. Architecture Overview

- The **Master database** is the **single source of truth** — all **write operations** (insert, update, delete) are directed here.
- The **Slave databases** are read-only replicas — they handle **read operations** to reduce load on the master.
- This improves:
 - **Availability**: Queries are spread across multiple read replicas.
 - **Latency**: Users are directed to the nearest slave.
 - **Reliability**: System remains operational even if some read replicas go down.

4. Data Synchronization / Replication

- The **Master** replicates its data to all **Slaves** using a process known as **Database Replication**.
- Replication can be:
 - **Synchronous**: Slaves must confirm updates before the master commits.
 - **Asynchronous**: Master commits immediately, and slaves catch up later.